

|                                                                     |    |
|---------------------------------------------------------------------|----|
| Liste linéaire doublement chaînée .....                             | 3  |
| Introduction .....                                                  | 3  |
| Définition .....                                                    | 4  |
| Représentation graphique.....                                       | 4  |
| Différences par rapport à une liste simplement chaînée.....         | 5  |
| Déclaration.....                                                    | 6  |
| Primitives de base : .....                                          | 6  |
| Algorithmes de Manipulation d'une liste doublement chaînée .....    | 8  |
| Création (Initialisation) d'une liste.....                          | 8  |
| Tester si une liste est vide ou pas.....                            | 8  |
| Tester si une liste est pleine ou pas.....                          | 9  |
| Insertion en tête.....                                              | 12 |
| Insertion en queue .....                                            | 14 |
| Insertion d'un élément Avant un autre élément.....                  | 17 |
| Insertion d'un élément Après un autre élément .....                 | 19 |
| Suppressions .....                                                  | 21 |
| Principe .....                                                      | 21 |
| Suppression en tête.....                                            | 22 |
| Suppression en queue .....                                          | 23 |
| Suppression autre que la tête et la queue.....                      | 24 |
| Amélioration : double adresses.....                                 | 25 |
| Primitives de base : .....                                          | 26 |
| Initialisation d'une liste .....                                    | 27 |
| Tester si une liste est vide ou pas.....                            | 27 |
| Insertion dans une liste doublement chaînée vide.....               | 28 |
| Insertion en tête dans une liste doublement chaînée quelconque..... | 31 |

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| Insertion en queue dans une liste doublement chaînée non vide.....    | 32 |
| Insertion Avant un élément dans une liste non vide .....              | 33 |
| Algorithmes de suppression.....                                       | 37 |
| Suppression en tête.....                                              | 38 |
| Suppression en queue .....                                            | 39 |
| Suppression autre que la tête et la queue.....                        | 40 |
| Suppression d'un élément donné par sa position ou par sa valeur ..... | 41 |

## LISTE LINÉAIRE DOUBLEMENT CHAÎNÉE

### INTRODUCTION

Dans le cas d'une **représentation contiguë** d'une liste linéaire (nommé tableau), on peut effectuer un parcours du tableau dans les deux sens (Gauche->Droite et Droite →Gauche).

Alors que, dans le cas d'une **représentation simplement chaînée**, on ne peut effectuer qu'un parcours de gauche à droite.

On peut qualifier cette liste linéaire chaînée de **monodirectionnelle**.

Si on veut pouvoir effectuer un parcours de droite à gauche, il faut ajouter un chaînage supplémentaire (càd un pointeur) permettant l'accès à la cellule précédente.

En qualifié alors la liste de **bidirectionnelle**.

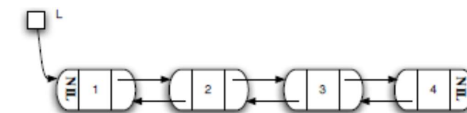
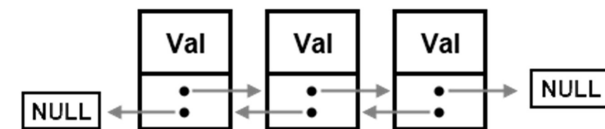
### Remarque

Compte de la place supplémentaire occupée, on n'utilise cette possibilité que dans des cas où on a très souvent besoin d'effectuer un retour en arrière.

## DÉFINITION

Chaque cellule de la liste dispose de deux adresses (=pointeurs) pointant respectivement vers la cellule précédente et la cellule suivante.

### Représentation graphique



Le pointeur supplémentaire (précédent) alourdit la structure.

Néanmoins, cette adjonction offre plus de flexibilité dans les modalités de parcours de la liste, qui peut maintenant être traversée en arrière.

## Différences par rapport à une liste simplement chaînée

- Parcours dans les deux sens (revenir en arrière)
- Permet de gagner en complexité sur le parcours, l'insertion et la suppression par rapport à une liste simplement chaînée

### Définition itérative

Une liste doublement chaînée est composée d'un nombre fini de cellules.

## Déclaration de la liste

```
typedef struct cellule{  
    TElement donnee;  
    struct cellule* suivant;  
    struct cellule* precedent;  
}*ListeDC;
```

### Primitives de base :

Fonction donnee : ListeDC  $\rightarrow$  TElement

Fonction suivant : ListeDC  $\rightarrow$  ListeDC

Fonction precedent : ListeDC  $\rightarrow$  ListeDC

## Opérations sur les listes doublement chaînées

- Création d'une liste vide (=init)
- Tester si une liste **est vide**
- Tester si une liste **est pleine**
- **Insertions** : tête, queue, avant élément, après élément, par position, etc
- **Suppressions** : tête, queue, avant élément, après élément, par position, etc
- **Parcours** (Affichage, Recherche, Somme, Modification,...)

## ALGORITHMES DE MANIPULATION D'UNE LISTE DOUBLEMENT CHAÎNÉE

### Création (Initialisation) d'une liste

Fonction initLDC : [] → ListeDC

Fonction initLDC() → ListeDC

Debut

Renvoyer NULL

Fin

### Tester si une liste est vide ou pas

Fonction estVideLDC : ListeDC → Booléenne

Fonction estVideDC(l) → Booléenne

P.F l : ListeDC (E )

Début

Renvoyer l = null

FIN

## Tester si une liste est pleine ou pas

Fonction estPleinLDC : ListeDC  $\rightarrow$  Booléenne

On suppose qu'une liste n'est jamais pleine

Fonction estPleinLDC(l)  $\rightarrow$  Booléenne

P.F l : ListeDC (E )

Début

Renvoyer faux

FIN

## Insertion dans une liste doublement chaînée

### Principe :

- Allocation mémoire & Remplissage
- Détermination de l'Adresse d'insertion
- Réalisation des chaînages

### Allocation Mémoire d'une nouvelle cellule & remplissage

Cette fonction permet d'allouer de l'espace mémoire pour une nouvelle cellule.

La fonction retournera l'adresse de la cellule nouvellement créée.

### Fonction allocRempCelLDC : TElement $\rightarrow$ ListeDC

Fonction allocRempCel(elt)  $\rightarrow$  ListeDC

P.F elt : TElement (E )

Debut

cel  $\leftarrow$  allocMemCellule()

//tester l'allocation mémoire

cel.donnee  $\leftarrow$  elt

renvoyer cel

Fin

```

ListeDC allocRempCel(TElement elt){
ListeDC cel;

    cel=(ListeDC)malloc(sizeof(struct cellule);

    if(!estVideDC(cel)){

        Cel->donnee=elt;

        return cel;

    }

    printf(« pb alloc mémoire ») ;

    exit(1) ;

}

```

## Insertion en tête

### Insertion d'un élément en tête de liste

inserTeteLDC : TElement X ListeDC --> ListeDC

#### Principe

- Allocation Mémoire & remplissage

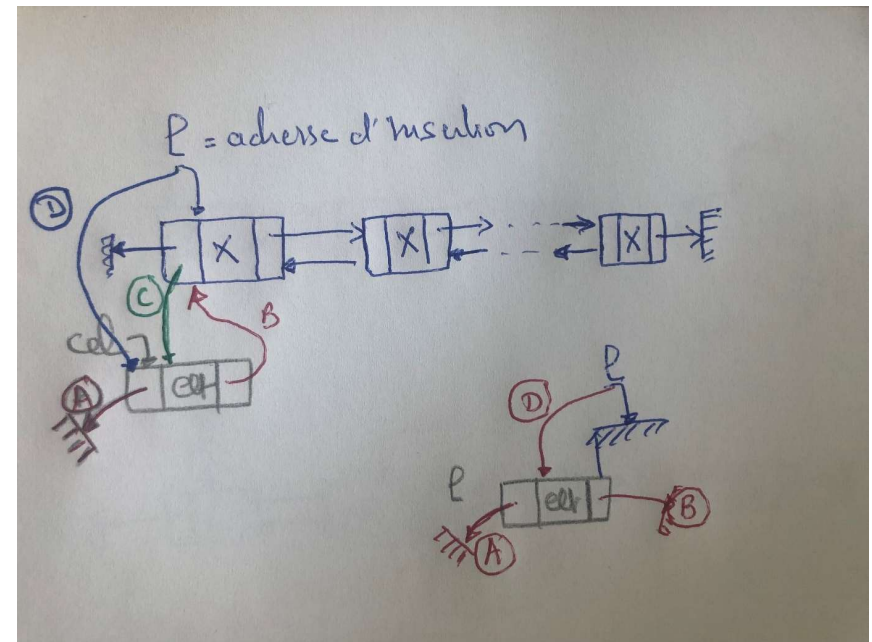
- Adresse d'insertion c'est l'adresse de la liste

- Chaînages

#### Principe par l'exemple

Cas où la liste est vide ➡ Idc ← allocRempCel(elt)

Cas où la liste est non vide :



Procédure inserTeteLDC(elt, l)

P.F elt : TElement (E )

l : ListeDC (E/S)

Debut

//alloc Mem et remplissage

cel ← allocRempCel(elt)

// réalisation des chainages

cel.precedent ← initL() //Chainage A

Si non estVideLDC(l) alors

cel.suivant ← l //Chainage B

l.precedent ← cel //Chainage C

l ← cel //Chainage D

Sinon

cel.suivant ← initL() //Chainage B

l ← cel

fsi

Fin

## Insertion en queue

## Insertion d'un élément en queue de liste

inserQueueLDC : TElement X ListeDC--> ListeDC

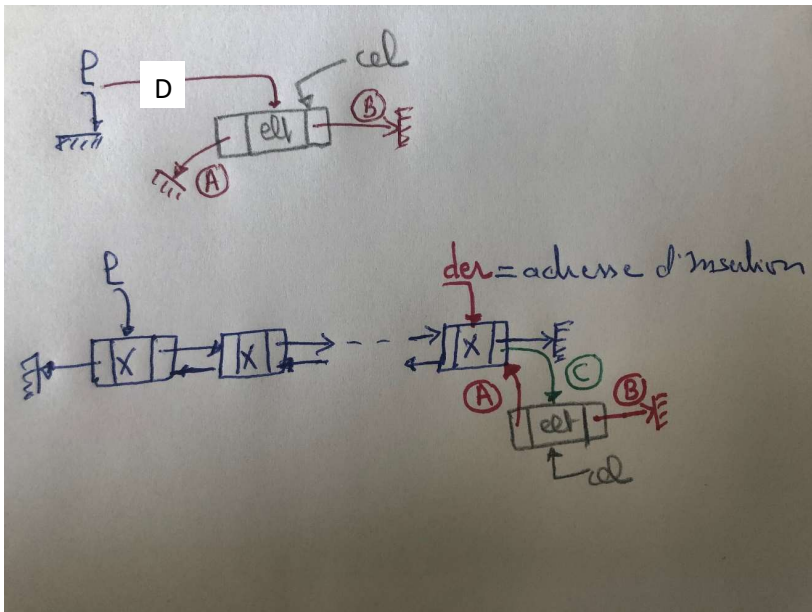
### Principe

- Si la liste est vide, on est ramené à une insertion en tête de liste (ou création d'une nouvelle cellule qui est aussi l'adresse de la liste)
- On procède comme suit si la liste n'est pas vide :
  - ❖ Déterminer l'adresse d'insertion.
  - ❖ Création d'une nouvelle cellule & Remplissage du membre donné de la cellule.
  - ❖ Réalisation des chaînages.

**Exemple 2 :****Cas où la liste est non vide :**

Il faut connaître l'adresse de la dernière cellule « **der** » .

On suppose que dernier est une fonction qui renvoie l'adresse de la dernière cellule d'une liste.

**L'algorithme d'insertion en Queue est alors****Procédure inserQueueLDC(elt, I)**

P.F elt : TELément (E)      I : listeDC (E/S)

Début

// Alloc Mem & Remplissage

cel ← allocRempCel(elt)

// Réalisation des chainages

cel.suivant ← initL()      //B

Si estVideLDC(I) alors

cel.précédent ← initL()      // A

I ← cel      //D

Sinon

**der ← dernier(I)**

cel.précédent ← der //chainage A

der.suivant ← cel //chainage C

Finsi

Fin



## Insertion d'un élément Avant un autre élément

Fonction inserAvant :

TElementxTElementxListeDC → ListeDC

### Principe

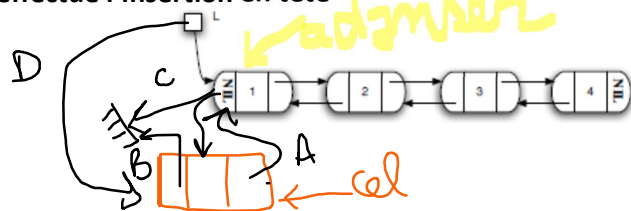
\* Recherche l'adresse de l'élément « eltAv »

\* Si cette adresse existe :

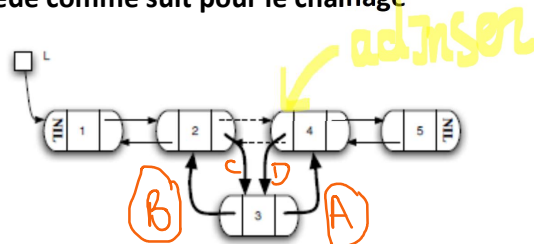
- Alloc Mémoire & Remplissage
- Modification des chaînages

### Exemple

Cas où l'adresse d'insertion est aussi l'adresse de la liste → Alors on effectue l'insertion en tête



Sinon on procède comme suit pour le chaînage



Procédure inserAvant(elt, eltAv, l)

P.F elt, eltAv : TElement (E)

l : ListeDC (E/S)

Debut

adInser ← adrInsertEltAv(eltAv, l)

//Vérifier l'existence de l'adr-Insertion

si non estVideLDC(adInser) alors

cel ← allocRempCel(elt)

cel.suivant ← adInser // A

cel.precedent ← precedent(adInser) // B

si adInser ≠ l alors

adInser.precedent.suivant ← cel // D

sinon

l ← cel // D

fsi

adInser.precedent ← cel // C

Fsi

Fin

## Insertion d'un élément Après un autre élément

Fonction inserApres :

TElementxTElementxListeDC → ListeDC

### Principe

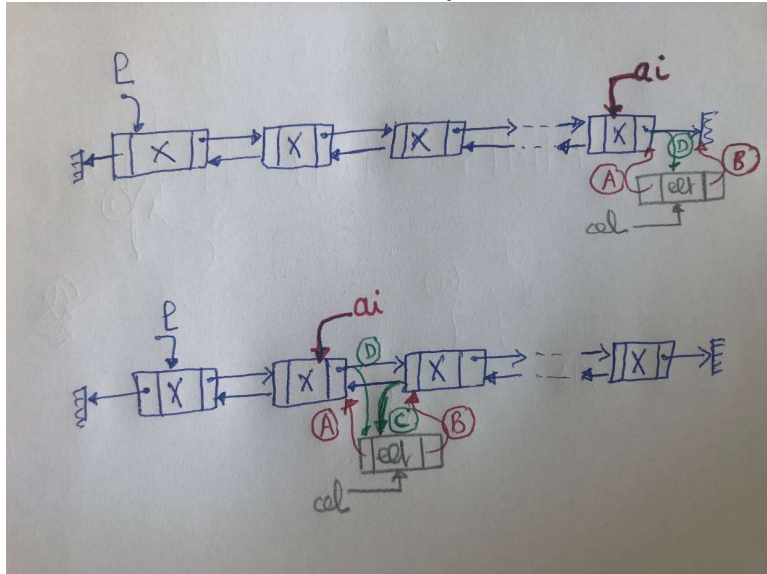
\* Recherche l'adresse de l'élément « eltAp »

\* Si cette adresse existe :

- Alloc Mémoire & Remplissage
- Modification des chaînages

### Exemple

Cas où l'adresse d'insertion est aussi l'adresse de la queue de la liste  
→ Alors on effectue l'insertion en queue



Procédure inserApres(elt, eltAp, l)

P.F elt, eltAv : TElement (E)

L : Liste DC (E/S)

Debut

adInser ← adrInsertEltAp(eltAp, l)

//Vérifier l'existence de l'adr-Insertion

si non estVideLDC(adInser) alors

cel ← allocRempCel(elt)

svAdInser ← suivant(adInser)

cel.precedent ← adInser //A

cel.suivant ← svAdInser //B

si non estVideLDC(svAdInser) alors //C

adInser.suivant.precedent ← cel

fsi

adInser.suivant ← cel //D

Fsi

Fin

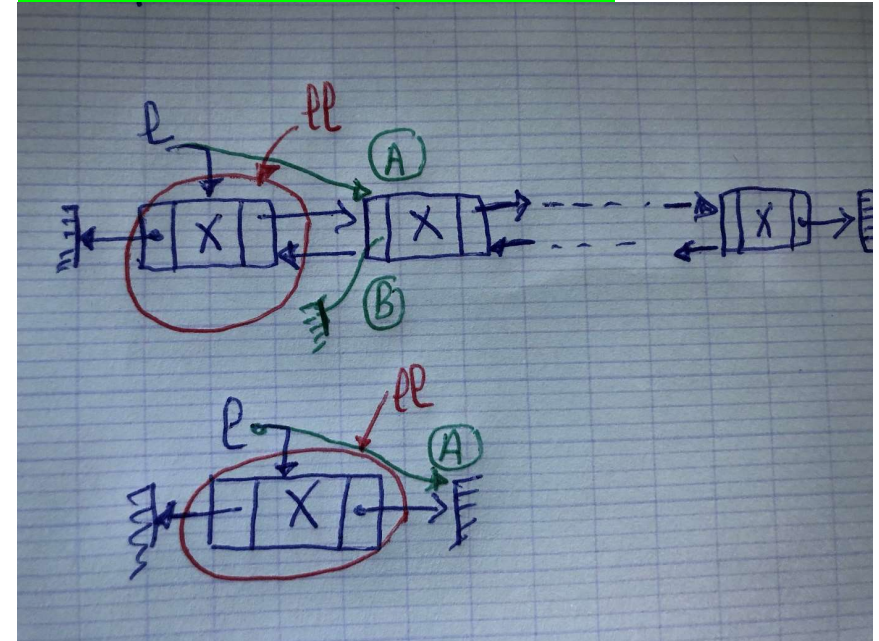
## SUPPRESSIONS

## Principe

- Déterminer l'adresse de suppression.
- Si cette adresse existe alors :
  - Repérer la cellule à supprimer.
  - Modification des chaînages.
  - Libération mémoire.

## Suppression en tête

L'adresse de suppression est l'adresse de la liste.



```
ListeDC supTeteLDC(ListeDC l){
```

```
ListeDC adSup ;
```

```
adSup = l ; //adresse de suppression
```

```
l = suivant(l) ; //chaînage (A)
```

```
If( !estVideLDC(l))
```

```
l->precedent = initL() ; //chainage (B)
```

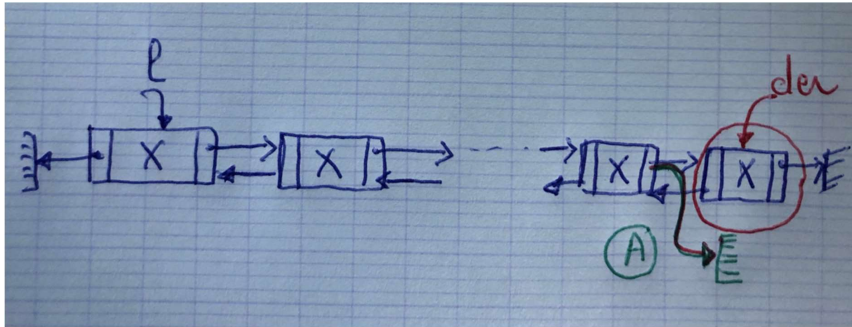
```
free(adSup);
```

```
return l;
```

```
}
```

## Suppression en queue

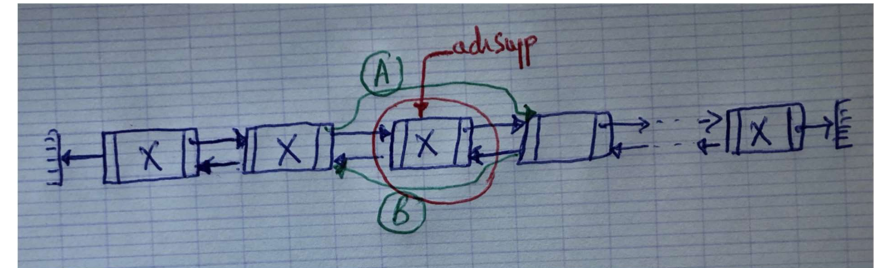
L'adresse de suppression est l'adresse de la dernière cellule de la liste.



```
ListeDC suppQueueLDC(ListeDC l){
ListeDC der ;
    der = dernier(l);           // adr de la dernière cellule
    (der->precedent)->suivant = initL() ; //chaînage (A)
    free(der);
    return l;
}
```

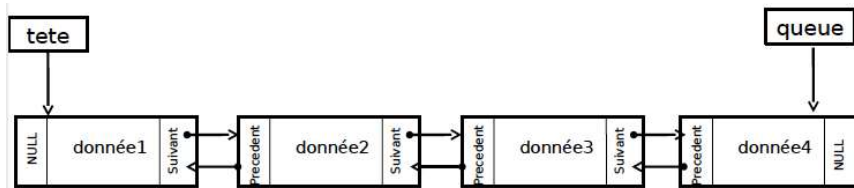
## Suppression autre que la tête et la queue

On suppose que **adSupp** est l'adresse de suppression ; différente de l'adresse de la tête et de l'adresse de la queue.



```
ListeDC suppAutreLDC(ListeDC l, ListeDC adSupp){
ListeDC cel ;
    cel=adSupp ;
    (adSupp->precedent)->suivant=suivant(adSupp);
    (adSupp->suivant)->precedent=precedent(adSupp);
    free(cel);
    return l;
}
```

## AMÉLIORATION : DOUBLES ADRESSES



Pour faciliter la mise à jour en queue ; nous représentons la liste doublement chaînée comme suit :

```
typedef struct cellule{
    TElement donnee;
    struct cellule* suiv;
    struct cellule* pred;
}*ListeDC;

typedef struct{
    [int taille;]

    ListeDC tete;

    ListeDC queue;
}ListeDCTQ;
```

## Primitives de base :

Fonction donnee : ListeDC  $\rightarrow$  TElement

Fonction suivant : ListeDC  $\rightarrow$  ListeDC

Fonction precedent : ListeDC  $\rightarrow$  ListeDC

[Fonction taille : ListeDCTQ  $\rightarrow$  Entier]

Fonction tete : ListeDCTQ  $\rightarrow$  ListeDC

Fonction queue : ListeDCTQ  $\rightarrow$  ListeDC

## Algorithmes de Manipulation d'une liste doublement chaînée

Nous allons maintenant créer des fonctions nous permettant de réaliser plusieurs opérations sur cette fameuse liste double chaînée.

### Initialisation d'une liste

Fonction initLDCTQ : [] → ListeDCTQ

Fonction initLDCTQ() → ListeDCTQ

Debut

l.tete ← NULL

l.queue ← NULL

l.taille ← 0

Renvoyer l

Fin

### Tester si une liste est vide ou pas

Fonction estVideL : ListeDCTQ → Booléenne

Fonction estVideDCTQ(l) → Booléenne

P.F l : ListeDC (E )

Début

Renvoyer taille(l) = 0

FIN

## Insertion dans une liste doublement chaînée vide

### Principe :

- Allocation mémoire & Remplissage
- Réalisation des chaînages

### Allocation Mémoire d'une nouvelle cellule & remplissage

Cette fonction permet d'allouer de l'espace mémoire pour une nouvelle cellule.

La fonction retournera l'adresse de la cellule nouvellement créée.

Fonction allocMemCelDC : TElement → ListeDC

Fonction allocMemCelDC (elt) → ListeDC

P.F elt : TElement (E )

Debut

cel ← allocMemCellule()

// vérifier l'alloc mémoire

cel.donnee ← elt

renvoyer cel

Fin

### Fonction inserLVideDCTQ : TElement → ListeDCTQ

Fonction inserLVideDCTQ(elt)

P.F elt : TElement (E )

Debut

cel ← allocMemCelDC(elt)

cel.suivant ← null

cel.précédent ← null

l.tete ← cel

l.queue ← cel

l.taille ← 1

renvoyer l

Fin

### Insertion en tête dans une liste doublement chaînée non vide

#### Fonction inserTeteL : TElement x ListeDCTQ → ListeDCTQ

#### Principe :

- Allocation mémoire & Remplissage
- Adresse d'insertion est l'adresse de la tête
- Réalisation des chaînages

#### procedure inserTeteLDCTQ(elt, l)

P.F elt : TElement (E )

l : ListeDCTQ (E/S )

Debut

cel ← allocMemCelDC(elt)

cel.suivant ← tete(l)

cel.precedent ← null

l.tete.precedent ← cel

l.tete ← cel

l.taille ← taille(l)+1

Fin

## Insertion en tête dans une liste doublement chaînée quelconque

Fonction `inserTeteL` : `TElement x ListeDCTQ → ListeDCTQ`

### Principe :

- Allocation mémoire & Remplissage
- Adresse d'insertion est l'adresse de la tête
- Réalisation des chaînages

**procedure** `inserTeteL(elt, l)`

P.F elt : `TElement (E)`      l : `ListeDCTQ (E/S)`

Debut

cel ← `creatCelluleLDC(elt)`

cel.precedent ← null

cel.suivant ← `tete(l)`

l.tete.precedent ← cel

l.tete ← cel

si `estVideL(l)` alors

    l.queue ← cel

fsi

l.taille ← `taille(l)+1`

Fin

## Insertion en queue dans une liste doublement chaînée non vide

Fonction `inserQueueL` : `TElement x ListeDCTQ → ListeDCTQ`

### Principe :

- Allocation mémoire & Remplissage
- Adresse d'insertion est celle de la queue
- Réalisation des chaînages

**procedure** `inserQueueL(elt, l)`

P.F elt : `TElement (E)`

l : `ListeDCTQ (E/S)`

**Debut**

cel ← `allocMemCeLDC(elt)`

cel.suivant ← null

cel.precedent ← `queue(l)`

l.queue.suivant ← cel

l.queue ← cel

l.taille ← `taille(l)+1`

**Fin**



## Insertion Avant un élément dans une liste non vide

Fonction `inserAvant` : `TElement x TElement x ListeDCTQ → ListeDCTQ`

### Principe

- \* Recherche l'adresse de l'élément « eltAv »
- \* Si cette adresse existe :
  - Allocation Mémoire & Remplissage
  - Modification des chaînages

### Exemple

```

Procédure inserAvant(elt, eltAv, l)
P.F elt, eltAv : TElement (E)
  l : ListeDCTQ (E/S)
Debut
  adInser ← adrInsertEltAvL(eltAv, l)
  si non estVideLDC(adInser) alors
    cel ← allocMemCelDC(elt)
    cel.suivant ← adInser
    cel.precedent ← precedent(adInser)
    Si adInser ≠ tete(l) alors
      adInser.precedent.suivant ← cel
  Sinon
    l.tete ← cel
  Fsi
  adInser.precedent ← cel
  l.taille ← taille(l)+1
Fsi
Fin
  
```

## Insertion Après un élément dans une liste **non vide**

Fonction `inserApres` : `TElement x TElement x ListeDCTQ → ListeDCTQ`

### Principe

- \* Recherche l'adresse de l'élément « eltAp »
- \* Si cette adresse existe :
  - Allocation Mémoire & Remplissage
  - Modification des chaînages

### Exemple

```

Procédure inserApres(elt, eltAp, l)
P.F elt, eltAp: TElement (E)
  l : ListeDCTQ (E/S)
Debut
  adInser ← adrInsertEltApL(eltAp, l)
  Si non estVideLDC(adInser) alors
    svAdInser ← suivant(adInser)
    cel ← allocMemCelDC(elt)
    cel.precedent ← adInser
    cel.suivant ← svAdInser
    Si non estVideLDC(svAdInser) alors
      adInser.suivant.precedent ← cel
    Sinon
      l.queue ← cel
  Fsi
  adInser.suivant ← cel
  l.taille ← taille(l)+1
Fsi
Fin
  
```

## ALGORITHMES DE SUPPRESSION

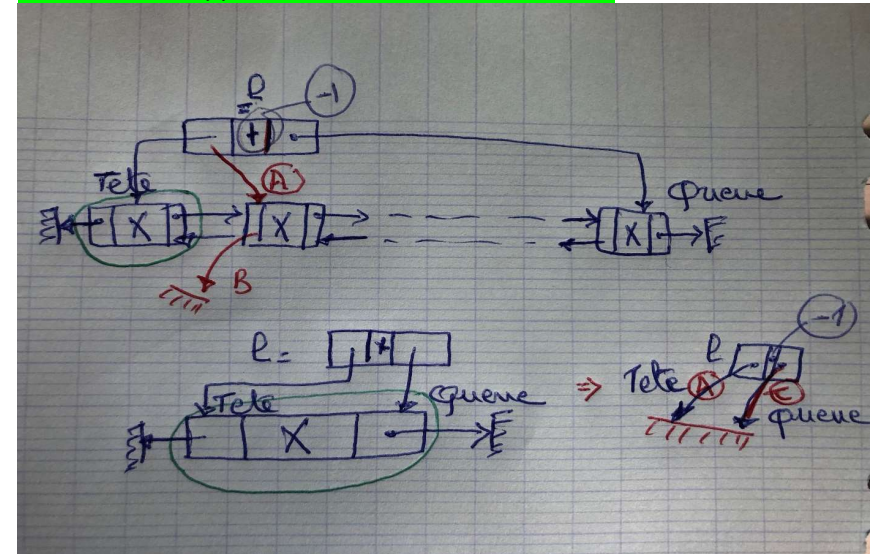
Hypothèse : on suppose que la liste est non vide

On distingue plusieurs cas de suppression

- Suppression en tête.
- Suppression en queue.
- Suppression de l'unique élément de la liste.
- Suppression de la liste.
- Suppression d'un élément donné par sa position.
- Suppression d'un élément donné par sa valeur dans une liste quelconque.
- Suppression d'un élément donné par sa valeur dans une liste triée.

## Suppression en tête

L'adresse de suppression est l'adresse de la liste.



```
ListeDC supTeteLDCTQ(ListeDCTQ l){
```

```
ListeDCTQ adSup ;
```

```
    adSup = tete(l) ; //adresse de suppression
```

```
    l.tete = suivant(tete(l)) ; //chaînage (A)
```

```
    (l.tete)->precedent = initL() ; //chaînage (B)
```

```
    If( estVideLDC(l))
```

```
        l.queue = initL() ; //chaînage (C)
```

```
    free(adSup);
```

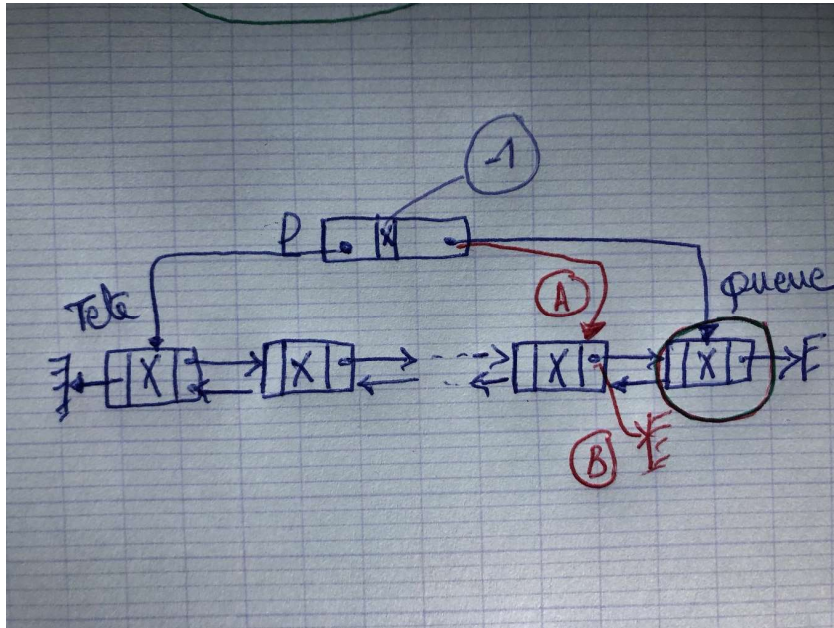
```
    l.taille =taille(l)-1 ;
```

```
    return l;
```

```
}
```

## Suppression en queue

L'adresse de suppression est l'adresse la queue de la liste.



```
ListeDCTQ suppQueueLDCTQ(ListeDCTQ l){
```

```
ListeDCTQ adSupp ;
```

```
    adSupp = queue(l);           // adresse de suppression
```

```
    l.queue= precedent(queue(l)); //chaînage (A)
```

```
    (l.queue)->precedent=iniL() ; //chaînage (B)
```

```
    free(adSupp);
```

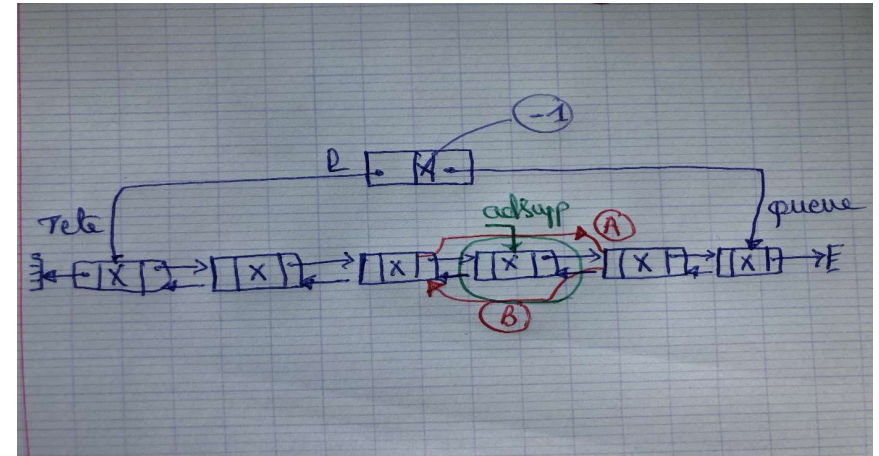
```
    l.taille =taille(l)-1 ;
```

```
    return l;
```

```
}
```

## Suppression autre que la tête et la queue

On suppose que **adSupp** est l'adresse de suppression ; différente de l'adresse de la tête et de l'adresse de la queue.



```
ListeDCTQ suppAutreLDCTQ(ListeDCTQ l, ListeDCTQ adSupp){
```

```
    (adSupp->precedent)->suivant=suivant(adSupp);
```

```
    (adSupp->suivant)->precedent=precedent(adSupp);
```

```
    free(adSupp) ;
```

```
    l.taille=taille(l)-1 ;
```

```
    return l ;
```

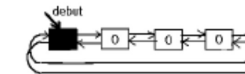
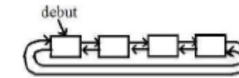
```
}
```

## Suppression d'un élément donné par sa position ou par sa valeur

Soit adSupp l'adresse de suppression

```
ListeDCTQ suppAutreLDC(ListeDCTQ l, ListeDCTQ adSupp){
    If(estVideLDC(precedent(adSupp)))
        return suppTeteLDCTQ(l) ;
    If(estVideLDC(precedent(adSupp)))
        return suppQueueLDCTQ(l) ;
    return suppAutreLDCTQ(l, adSupp) ;
}
```

## Variantes



etc